

REVISÃO PARA A AVALIAÇÃO

Componentes, props, layout, interação, estado e formulários

Análise e Desenvolvimento de Sistemas

Tecnologia usada em aula: **React Native + Expo**

O QUE ESTA REVISÃO COBRE

Quatro blocos, um app como fio condutor

- 1 **Componentes e props** — quebrar a tela em peças reutilizáveis
- 2 **Layout, estilo e conteúdo visual** — `StyleSheet`, `Flexbox`, `Text`, `Image`
- 3 **Interação e estado** — toque, `useState` e `Alert`
- 4 **Entrada de dados** — `TextInput`, teclado e validação

Todo exemplo desta revisão saiu do app **profile**, que montamos em aula. Se você entende aquele arquivo inteiro, você domina a matéria.

O APP QUE AMARRA TUDO

A tela de perfil, montada em aula

```
+-----+
|          Perfil          | <Text> no topo
+-----+
|      +-----+          |
|      | avatar |          | <TouchableOpacity> + <Image>
|      +-----+          |
|      Alterar foto        |
|  [ Nome                  ] <TextInput> editavel
|  [ E-mail                ] <TextInput> desabilitado
|  Alterar senha           |
|  [ Senha antiga          ] secureTextEntry
|  [ Nova senha            ]
|  [      Salvar           ] <Button /> proprio
+-----+
```

OS ARQUIVOS DO PROJETO

Onde cada coisa mora

profile/	
App.js	a tela inteira: layout, estado e campos
index.js	registra o App (nao mexemos nele)
app.json	nome, icone e configuracoes do app
package.json	dependencias do projeto
src/	
components/	
Button.js	nosso componente reutilizavel
assets/	imagens e icones

Regra de organização: **uma tela por arquivo**; todo pedaço reutilizável desce para src/components .

PARTE 1

Componentes e props

O QUE É UM COMPONENTE

Uma função JavaScript que devolve JSX

Não existe magia: é uma função comum. O que a torna um componente é **devolver JSX** e ser usada como uma tag.

```
function Saudacao() {  
  return <Text>Olá!</Text>;  
}
```

```
// usar é escrever a função como se fosse uma tag  
<Saudacao />
```

JSX

O React chama essa função sempre que precisa desenhar (ou redesenhar) aquele pedaço da tela.

AS QUATRO REGRAS DO COMPONENTE

Elas explicam quase todo erro de sintaxe

- 1 O nome **sempre** começa com letra maiúscula — `Button`, não `button`
- 2 O `return` devolve **um único elemento pai**
- 3 Todo elemento precisa ser fechado, inclusive `<Image />` e `<Button />`
- 4 Todo texto visível fica dentro de um `<Text>`

Minúscula não dá erro claro: o React acha que é um elemento nativo e a tela some.

EXPORTAR E IMPORTAR

`export default` de um lado, `import` do outro

```
// src/components/Button.js
export default function Button({ title, onPress, disabled }) { ... }
```

JSX

```
// App.js – nome livre, sem chaves, caminho relativo
import Button from './src/components/Button';

// import nomeado: nome exato, entre chaves
import { StyleSheet, Text, View } from 'react-native';
```

JSX

`export default` = uma coisa principal por arquivo. Import **sem chaves** pega o default; **com chaves** pega os nomeados.

PROPS: O QUE SÃO

Os parâmetros do componente

Props são os dados que o **pai** passa para o **filho**. Chegam como um objeto, na ordem em que você escreveu os atributos.

```
<Button title="Salvar" onPress={submitForm} disabled={true} />
//      ^ string      ^ função      ^ booleano
```

JSX

- Texto puro vai entre **aspas**: `title="Salvar"`
- Qualquer outro valor vai entre **chaves**: número, booleano, função, objeto, array
- Props são **somente leitura**: o filho nunca altera o que recebeu

O NOSSO **BUTTON**, INTEIRO

O componente que você precisa saber explicar

JSX

```
import { StyleSheet, Text, TouchableOpacity } from "react-native";

export default function Button({ title, onPress, disabled }) {
  return (
    <TouchableOpacity
      style={styles.btnSubmit, { opacity: disabled ? 0.3 : 1 }}
      onPress={onPress}
      disabled={disabled}
    >
      <Text style={styles.label}>{title}</Text>
    </TouchableOpacity>
  );
}
```

BUTTON — O QUE CADA PROP FAZ

Três props, três papéis diferentes

PROP	TIPO	O QUE FAZ
<code>title</code>	string	vira o texto dentro do <code><Text></code>
<code>onPress</code>	função	é disparada quando o usuário toca
<code>disabled</code>	booleano	bloqueia o toque e reduz a opacidade

Note que `disabled` faz duas coisas: passa para o `TouchableOpacity` (comportamento) e entra no array de estilo (aparência).

DESESTRUTURAÇÃO DAS PROPS

Duas escritas para a mesma coisa

```
// forma longa: props é um objeto  
export default function Button(props) {  
  return <Text>{props.title}</Text>;  
}
```

JSX

```
// forma usada em aula: já abre o objeto na assinatura  
export default function Button({ title, onPress, disabled }) {  
  return <Text>{title}</Text>;  
}
```

JSX

As chaves na assinatura **não** são um objeto novo: são desestruturação. Esquecê-las é o erro clássico — `title` fica `undefined`.

VALOR PADRÃO DE UMA PROP

Para a prop que nem sempre é passada

```
export default function Button({ title = "Enviar", onPress, disabled = false }) {  
  ...  
}
```

JSX

```
<Button onPress={salvar} />           // usa "Enviar" e disabled false  
<Button title="Salvar" onPress={salvar} />
```

JSX

Sem valor padrão, uma prop não passada chega como `undefined` — e `undefined` dentro de um `<Text>` não desenha nada.

PROPS BOOLEANAS: O ATALHO

`secureTextEntry` e `secureTextEntry={true}` são a mesma coisa

```
<TextInput secureTextEntry={true} />  
<TextInput secureTextEntry />           // idêntico ao de cima
```

JSX

Os dois campos de senha do `profile` usam as duas escritas de propósito, para mostrar que são equivalentes.

Escrever a prop **sem valor** significa `true`. Para `false`, você é obrigado a escrever `= {false}`.

PASSAR UMA FUNÇÃO POR PROP

A armadilha mais comum da prova

JSX

```
<Button title="Salvar" onPress={submitForm} /> // certo: passa a função  
<Button title="Salvar" onPress={submitForm()} /> // errado: chama na hora
```

ARMADILHA Com os parênteses, a função roda **durante a renderização** — o `Alert` aparece sozinho ao abrir a tela, e o toque não faz nada.

QUANDO É PRECISO PASSAR ARGUMENTO

Aí sim entra a arrow function

JSX

```
// precisa de argumento? embrulhe em uma função  
<Button title="Apagar" onPress={() => remover(item.id)} />  
  
// sem argumento? passe a referência direto  
<Button title="Salvar" onPress={submitForm} />
```

`() => remover(item.id)` é uma função que, **quando chamada**, chama `remover`. É por isso que ela não dispara sozinha.

CHILDREN E COMPOSIÇÃO

O conteúdo que vai entre as tags

```
export default function Card({ children }) {  
  return <View style={styles.card}>{children}</View>;  
}
```

JSX

```
<Card>  
  <Text>Ana Souza</Text>  
  <Text>Desenvolvedora mobile</Text>  
</Card>
```

JSX

children é uma prop automática: tudo o que estiver **entre** a tag de abertura e a de fechamento chega por ela.

POR QUE EXTRAIR UM COMPONENTE

O mesmo botão, antes e depois

JSX

```
// ANTES: dentro do App.js
<TouchableOpacity
  style={styles.btnSubmit}
  onPress={submitForm}
  disabled={name.length == 0}
>
  <Text style={styles.label}>
    Salvar
  </Text>
</TouchableOpacity>
```

JSX

```
// DEPOIS: um componente próprio
<Button
  title="Salvar"
  onPress={submitForm}
  disabled={name.length == 0}
/>
```

Menos linhas no `App.js`, estilo em um lugar só e o botão pronto para as próximas telas.

O QUE GANHAMOS AO EXTRAIR

Quatro argumentos que valem como resposta de prova

- **Reuso** — o mesmo botão em qualquer tela, sem copiar estilo
- **Manutenção** — mudou a cor? um arquivo só
- **Legibilidade** — o `App.js` passa a descrever a tela, não os detalhes
- **Teste e depuração** — o problema fica cercado em um arquivo pequeno

Regra prática: repetiu o mesmo bloco duas vezes, vire componente na terceira.

DOCUMENTAR O COMPONENTE

O bloco de comentário do nosso `Button.js`

```
/**
 * @param {string} title - texto que será renderizado dentro do btn
 * @param {function} onPress - função que o botão irá disparar
 * @param {boolean} disabled - condição para desabilitar o btn
 *
 * @example
 * <Button title="Texto" onPress={handleFunction} disabled={true} />
 */
```

JSX

Esse comentário aparece no autocomplete do VS Code quando você digita `<Button`. É o jeito barato de documentar um componente em JavaScript.

PARTE 2

Layout, estilo e conteúdo visual

NÃO EXISTE CSS AQUI

O estilo é um objeto JavaScript

Não existe arquivo `.css`, não existe seletor, não existe cascata e não existe `!important`.

- O estilo é um **objeto**, criado com `StyleSheet.create`
- Cada componente recebe o seu estilo pela prop `style`
- O React Native traduz esse objeto para propriedades nativas do Android e do iOS

A vantagem é direta: estilo é código. Dá para calcular, guardar em variável e reaproveitar.

AS QUATRO REGRAS DE ESCRITA

Todo erro de estilo do primeiro mês está nesta lista

- 1 Propriedade em **camelCase**: `backgroundColor`, não `background-color`
- 2 Medida é **número puro**: `fontSize: 16`, nunca `"16px"`
- 3 Palavra-chave é **string**: `"center"`, `"bold"`, `"cover"`
- 4 Porcentagem é **string**: `width: "50%"`

Número puro significa DP (pixel independente de densidade) — o sistema converte para o pixel real de cada aparelho.

CSS E REACT NATIVE LADO A LADO

Mesma ideia, outra grafia

CSS	REACT NATIVE
<code>background-color: #eee</code>	<code>backgroundColor: '#eee'</code>
<code>font-size: 16px</code>	<code>fontSize: 16</code>
<code>margin: 8px 16px</code>	<code>marginVertical: 8, marginHorizontal: 16</code>
<code>border: 1px solid #ccc</code>	<code>borderWidth: 1, borderColor: '#ccc'</code>
<code>display: flex</code>	já é o padrão

Atalhos com vários valores não existem: cada lado é uma propriedade.

POR QUE `STYLE SHEET.CREATE`

Não é enfeito

- **Valida** as propriedades: nome errado acusa erro em vez de ser ignorado
- Dá **autocomplete** e tipagem no VS Code
- O objeto é criado **uma vez**, e não a cada redesenho do componente
- Separa estrutura de aparência: o JSX fica legível

Estilo inline (`style={{ ... }}`) funciona, mas guarde-o para valores que **mudam em tempo de execução** — como a opacidade do nosso `Button` .

TRÊS FORMAS DE APLICAR ESTILO

Objeto único à esquerda, combinação à direita

```
<View style={styles.info}>
  <Text style={styles.label}>
    Alterar senha
  </Text>
</View>
```

JSX

```
<TextInput
  style={[styles.input, styles.inputDisabled]}
/>

<View style={[styles.card, { opacity: 0.5 }]} />
```

JSX

Um array de estilos é combinado da esquerda para a direita. Em caso de conflito, o último vence.

ARRAY DE ESTILOS NO PROFILE

Um estilo base e uma variação em cima dele

```
<TextInput
  style={[styles.input, styles.inputDisabled]}
  editable={false}
/>
```

JSX

```
input: {
  backgroundColor: '#39393C', height: 56, color: '#fff',
  borderRadius: 8, paddingLeft: 16, fontSize: 16,
},
inputDisabled: { backgroundColor: '#202024', color: '#C4C4CC' },
```

JSX

`inputDisabled` só reescreve **duas** propriedades; altura, raio e espaçamento continuam vindo de `input`.

ESTILO CONDICIONAL

O estilo reage ao estado, como qualquer outro valor

```
// no nosso Button.js  
style={[styles.btnSubmit, { opacity: disabled ? 0.3 : 1 }]}
```

JSX

```
// outra escrita, com E lógico  
style={[styles.base, ativo && styles.ativo]}
```

JSX

Valores `false`, `null` e `undefined` dentro do array são simplesmente **ignorados** — por isso o `&&` funciona ali.

ESTILO NÃO SE HERDA

A regra que mais confunde quem vem do CSS

- Cada componente só usa o estilo que recebeu na **própria** prop `style`
- Não existe cascata, não existe seletor descendente
- A única exceção: um `<Text>` dentro de outro `<Text>` herda a tipografia

ARMADILHA `<View style={{ color: 'red' }}>` não dá erro e não faz nada. O `color` precisa ir no `<Text>`.

A CAIXA: ESPAÇAMENTO E BORDA

As propriedades que aparecem em toda tela

PROPRIEDADE	O QUE FAZ
<code>padding</code>	espaço dentro da caixa
<code>margin</code>	espaço fora da caixa
<code>borderWidth</code> / <code>borderColor</code>	a linha da borda
<code>borderRadius</code>	arredonda os cantos
<code>overflow: 'hidden'</code>	corta o que passar do limite

Cada uma aceita variantes por lado: `paddingLeft`, `marginVertical`, `marginTop`, `borderTopLeftRadius`.

TUDO AQUI JÁ É FLEXBOX

Os três defaults que mudam em relação à web

PROPRIEDADE	WEB	REACT NATIVE
<code>flexDirection</code>	<code>row</code>	<code>column</code>
<code>alignContent</code>	<code>stretch</code>	<code>flex-start</code>
<code>flexShrink</code>	<code>1</code>	<code>0</code>

ATENÇÃO O padrão é **coluna**. Se você espera os elementos lado a lado e eles saem empilhados, faltou `flexDirection: 'row'`.

JUSTIFYCONTENT

Distribui os filhos ao longo do eixo principal

VALOR	RESULTADO
flex-start	tudo no início (padrão)
center	tudo no centro
flex-end	tudo no fim
space-between	primeiro na ponta, último na outra
space-around	espaço igual em volta de cada um
space-evenly	espaço igual entre todos e nas bordas

ALIGNITEMS

Alinha os filhos no eixo cruzado

VALOR	RESULTADO
stretch	filho ocupa toda a largura/altura (padrão)
flex-start	encosta no começo do eixo cruzado
center	centraliza no eixo cruzado
flex-end	encosta no fim
baseline	alinha pela linha de base do texto

O padrão é `stretch` : por isso uma `<View>` sem largura definida ocupa a linha inteira.

CENTRALIZAR DE VERDADE

A dupla que resolve a tela inteira

```
tela: {  
  flex: 1,           // ocupa a tela inteira  
  justifyContent: 'center', // centraliza na vertical  
  alignItems: 'center',   // centraliza na horizontal  
},
```

JSX

Sem `flex: 1`, a View tem só a altura do conteúdo e não sobra espaço nenhum para centralizar. É o motivo número um de "o `justifyContent` não faz nada".

FLEX: 1 — OCUPAR O QUE SOBRA

O número é uma proporção, não um tamanho

```
<View style={{ flex: 1, flexDirection: 'row' }}>
  <View style={{ flex: 1 }} />
  <View style={{ flex: 2 }} />
  <View style={{ flex: 1 }} />
</View>
```

JSX

| 25% | 50% | 25% |

Somam-se os valores ($1 + 2 + 1 = 4$) e cada filho recebe a sua fração do espaço livre.

ALIGNSELF E GAP

O filho que foge da regra, e o espaçamento entre irmãos

```
profile: { alignSelf: 'center', marginVertical: 24 }, // só este filho centraliza
btnAvatar: { alignItems: 'center', gap: 12 }, // 12 entre a foto e o texto
info: { gap: 16, marginTop: 32 }, // 16 entre os campos
```

JSX

- `alignSelf` sobrescreve o `alignItems` do pai, **só para aquele filho**
- `gap` cria espaço entre os filhos sem margem sobrando no último

Prefira `gap` a espalhar `marginBottom` em cada filho.

O LAYOUT DO PROFILE, CAMADA POR CAMADA

Três **View** aninhadas resolvem a tela inteira

```
container: {  
  flex: 1, // a tela toda  
  backgroundColor: '#39393C', // cinza da faixa do topo  
},  
main: {  
  flex: 1, // ocupa todo o espaço que sobrar  
  backgroundColor: '#121214', // painel escuro  
  paddingVertical: 24,  
  paddingHorizontal: 40,  
},  
info: { gap: 16, marginTop: 32 }, // cada grupo de campos
```

JSX

TEXT — TIPOGRAFIA

O conjunto que resolve quase tudo

PROPRIEDADE	EXEMPLO
fontSize	24
fontWeight	'500', 'bold'
color	'#fff'
textAlign	'center'
lineHeight	22 (valor absoluto em DP)
textTransform	'uppercase'

Texto solto dentro de uma `<View>` derruba o app na hora.

TEXTO QUE NÃO CABE

`numberOfLines` e `ellipsiMode`

JSX

```
<Text style={styles.cargo} numberOfLines={2} ellipsiMode="tail">  
  Desenvolvedora mobile, apaixonada por interfaces bem construídas  
</Text>
```

Desenvolvedora mobile, apaixonada por interfaces bem construídas e por café passado na...

`ellipsiMode` aceita "tail" (padrão), "head", "middle" e "clip". Sem uma largura limitada pelo pai, `numberOfLines` não trunca nada.

IMAGE: LOCAL OU REMOTA

Duas fontes, duas sintaxes

JSX

```
// local: require com caminho fixo
<Image
  source={require('./assets/icon.png')}
/>
```

JSX

```
// remota: objeto com uri
<Image
  source={{ uri: 'https://...' }}
  style={{ width: 180, height: 180 }}
/>
```

O `require` só aceita caminho literal: não funciona com variável nem com string montada.

A REGRA DA IMAGEM REMOTA

O erro mais comum do dia

Uma imagem vinda da internet chega **sem tamanho conhecido**. Sem `width` e `height` no estilo, ela é renderizada com altura zero — e você vê um espaço em branco.

- Imagem **local**: o tamanho vem do arquivo, o estilo é opcional
- Imagem **remota**: sempre declare `width` e `height` (ou use `flex: 1`)

ARMADILHA "Minha imagem não aparece" quase nunca é URL errada. É altura zero.

SAFEAREAVIEW E STATUSBAR

O entalhe, a câmera e a barra do sistema

```
import { SafeAreaView } from 'react-native-safe-area-context';
import { StatusBar } from 'expo-status-bar';

<SafeAreaView style={styles.container}>
  <StatusBar style="auto" />
  ...
</SafeAreaView>
```

JSX

- Sem `SafeAreaView`, o topo da tela fica escondido atrás do relógio e do entalhe
- `StatusBar`: `style="dark"` para fundo claro, `"light"` para fundo escuro, `"auto"` decide sozinho

DEPURANDO LAYOUT

O truque que economiza meia hora

JSX

```
const debug = { borderWidth: 1, borderColor: 'red' };  
  
<View style={[styles.main, debug]}>
```

- Coloque uma borda colorida temporária na `View` que está estranha
- Se um elemento sumiu, quase sempre ele tem **altura zero** — não cor errada
- Confira sempre nesta ordem: `flexDirection`, depois `justifyContent`, depois `alignItems`

PARTE 3

Interação e estado

OS CINCO COMPONENTES DE TOQUE

O que muda é o feedback visual

COMPONENTE	FEEDBACK AO TOQUE
<code>Button</code>	visual do sistema; quase não aceita estilo
<code>TouchableOpacity</code>	o conteúdo fica transparente
<code>TouchableHighlight</code>	um fundo colorido aparece atrás
<code>TouchableWithoutFeedback</code>	nenhum — só captura o toque
<code>Pressable</code>	você decide, pelo estado <code>pressed</code>

No celular não existe `onClick`: o evento é `onPress`.

BUTTON E SEUS LIMITES

Serve para o protótipo, não para o app

```
<Button title="Somar 1" onPress={() => setContador(contador + 1)} />
```

JSX

- Aceita basicamente `title`, `onPress`, `color` e `disabled`
- **Não** aceita `style`: você não controla altura, raio nem fonte
- No Android e no iOS ele tem aparências diferentes

É exatamente por isso que criamos o nosso próprio `Button` sobre um `TouchableOpacity`.

TOUCHABLEOPACITY

O mais usado no dia a dia

JSX

```
<TouchableOpacity  
  activeOpacity={0.5}  
  style={styles.btnAvatar}  
>  
  <Image source={{ uri: '...' }} style={styles.img} />  
  <Text style={styles.alterarTxt}>Alterar foto</Text>  
</TouchableOpacity>
```

`activeOpacity` é a opacidade **enquanto o dedo está pressionando** (padrão `0.2`). Repare que ele envolve **dois** filhos: a área tocável é a caixa inteira.

TOUCHABLEWITHOUTFEEDBACK

Captura o toque sem mostrar nada

```
function fecharTeclado() {  
  Keyboard.dismiss();  
}  
  
<TouchableWithoutFeedback onPress={fecharTeclado}>  
  <SafeAreaView style={styles.container}>  
    ...  
  </SafeAreaView>  
</TouchableWithoutFeedback>
```

JSX

É assim que o `profile` fecha o teclado ao tocar em qualquer lugar da tela. Ele aceita **um único filho**, e esse filho precisa ser um componente nativo.

PRESSABLE

O mais novo e o mais flexível

JSX

```
<Pressable
  onPress={salvar}
  onLongPress={apagar}
  hitSlop={8}
  style={({ pressed }) => [styles.btn, pressed && styles.btnPressed]}
>
  <Text style={styles.label}>Salvar</Text>
</Pressable>
```

No Pressable, style pode ser uma **função** que recebe { pressed }. É o único que deixa você desenhar o estado pressionado do seu jeito.

PROPS DE TOQUE QUE CAEM EM PROVA

Comportamento e acessibilidade

PROP	O QUE FAZ
<code>onPress</code>	dispara ao soltar o dedo
<code>onLongPress</code>	dispara ao segurar
<code>disabled</code>	ignora o toque
<code>activeOpacity</code>	opacidade durante o toque (<code>TouchableOpacity</code>)
<code>underlayColor</code>	cor do fundo durante o toque (<code>TouchableHighlight</code>)
<code>hitSlop</code>	aumenta a área tocável sem mudar o visual
<code>accessibilityLabel</code>	o que o leitor de tela anuncia

USESTATE : AS DUAS METADES

Um valor e a função que o troca

```
import { useState } from 'react';

const [name, setName] = useState('');
//      ^valor  ^funcao que troca  ^valor inicial
```

JSX

- O **valor** é só leitura: `name = 'Ana'` não faz nada
- A **função** avisa o React de que precisa redesenhar
- O valor inicial é usado apenas na **primeira** renderização

`useState` é um Hook: só pode ser chamado no topo do componente, nunca dentro de `if`, `laço` ou função aninhada.

O CICLO QUE VOCÊ PRECISA SABER EXPLICAR

Evento, estado, redesenho

```
usuario digita      setName('Ana')      o React chama
no <TextInput>      troca o estado      a funcao App()
                    --->                de novo
                    |                    |
                    |                    v
                    +-----+ a tela reflete o novo -----+
                           valor de name
```

Você **não** manda a tela mudar. Você muda o estado, e o React redesenha o componente inteiro com o valor novo.

NUNCA ALTERE O ESTADO NA MÃO

Imutabilidade é a regra

```
lista.push('Arroz');           // errado: o React nao percebe  
setLista([...lista, 'Arroz']); // certo: array novo
```

JSX

ARMADILHA `push`, `pop`, `splice` e `obj.campo = x` alteram o valor **no lugar**. Como a referência é a mesma, o React entende que nada mudou e a tela não atualiza.

ELEVAÇÃO DE ESTADO

O estado mora no pai; o filho recebe props

```
// App.js – o dono do estado  
const [name, setName] = useState('');  
  
<TextInput onChangeText={text => setName(text)} />  
<Button title="Salvar" onPress={submitForm} disabled={name.length == 0} />
```

JSX

- O `Button` **não tem estado**: ele só recebe `disabled` já calculado
- Quem sabe o que fazer no toque é o pai, que passa a função por `onPress`

Dados descem por props; eventos sobem por funções. Esse é o fluxo do React.

ALERT

A resposta imediata a uma ação

```
import { Alert } from 'react-native';

function submitForm() {
  return Alert.alert(`Olá, ${name}`);
}
```

JSX

```
// com botões e ação de confirmação
Alert.alert('Remover', 'Tem certeza?', [
  { text: 'Cancelar', style: 'cancel' },
  { text: 'Remover', onPress: () => remover(id) },
]);
```

JSX

Alert é do react-native. O alert() do navegador **não existe** aqui.

O ESTADO DO PROFILE, PONTA A PONTA

Um único `useState` comanda três coisas

```
const [name, setName] = useState('');

<TextInput onChangeText={(text) => setName(text)} />    // 1. escreve

<Button
  title="Salvar"
  onPress={submitForm}                                     // 2. usa: Alert com o nome
  disabled={name.length == 0}                             // 3. decide: botao ligado ou nao
/>
```

JSX

Enquanto name estiver vazio, disabled é true: o botão fica com opacidade 0.3 e não responde ao toque. Digitou uma letra, ele acende sozinho.

PARTE 4

Entrada de datos — TextInput

TEXTINPUT — O BÁSICO

O campo de texto do celular

```
import { TextInput } from 'react-native';

const [name, setName] = useState('');

<TextInput
  style={styles.input}
  placeholder="Nome"
  onChangeText={(text) => setName(text)}
/>
```

JSX

Sem `style`, o campo aparece sem altura, sem fundo e praticamente invisível. Estilo de `TextInput` **não** vem pronto.

ONCHANGETEXT DEVOLVE O TEXTO

A diferença que pega quem vem da web

```
// React Native – chega o texto direto  
onChangeText={(text) => setName(text)}  
  
// React web – chega o evento  
onChange={(e) => setName(e.target.value)}
```

JSX

ARMADILHA `e.target.value` não existe no React Native. Não há DOM, não há `event.target`.

CONTROLADO x NÃO CONTROLADO

`value` manda; `defaultValue` só inicia

JSX

```
// CONTROLADO
<TextInput
  value={name}
  onChangeText={setName}
/>
// o estado e a unica verdade:
// limpar o estado limpa o campo
```

JSX

```
// NAO CONTROLADO (o do profile)
<TextInput
  defaultValue={name}
  onChangeText={setName}
/>
// o campo guarda o proprio texto;
// o estado so acompanha
```

Com `value`, `setName('')` apaga o campo na tela. Com `defaultValue`, não apaga — o texto continua lá. Para formulário que precisa ser limpo, use `value`.

AS PROPS DO TEXTINPUT

O conjunto que resolve quase todo formulário

PROP	O QUE FAZ
placeholder	texto de dica quando está vazio
placeholderTextColor	cor dessa dica
value / defaultValue	conteúdo controlado / inicial
onChangeText	dispara a cada caractere
secureTextEntry	esconde o texto (senha)
editable	false bloqueia a digitação
keyboardType	qual teclado abre
multiline	permite várias linhas

KEYBOARDTYPE

O teclado certo para cada campo

VALOR	TECLADO
default	comum
email-address	com @ e ponto
numeric	só números
phone-pad	discagem
decimal-pad	números com vírgula

Em campo de e-mail, use sempre `keyboardType="email-address"` e `autoCapitalize="none"` — senão o celular escreve `Marcos@gmail.com` com maiúscula.

OS CAMPOS DO PROFILE

Três configurações diferentes na mesma tela

```
<TextInput
  style={styles.input}
  selectionColor="#00B37E"           // cor do cursor e da selecao
  placeholder="Nome"
  placeholderTextColor="#7C7C8A"
  defaultValue={name}
  onChangeText={(text) => setName(text)}
/>
```

JSX

`selectionColor` muda só o indicador de digitação. É detalhe visual, mas é o tipo de coisa que aparece na correção.

CAMPO DESABILITADO

`editable={false}` mais um estilo próprio

```
<TextInput
  style={[styles.input, styles.inputDisabled]}
  placeholder="E-mail"
  placeholderTextColor="#7C7C8A"
  defaultValue="marcos@gmail.com"
  editable={false}
/>
```

JSX

Bloquear a digitação é **comportamento**; deixar o campo mais escuro é **aparência**. As duas coisas precisam ser feitas: um campo bloqueado com aparência normal confunde o usuário.

CAMPO DE SENHA

`secureTextEntry` e as duas escritas

JSX

```
<TextInput
  style={styles.input}
  placeholder="Senha antiga"
  placeholderTextColor="#7C7C8A"
  secureTextEntry={true}
/>
```

```
<TextInput placeholder="Nova Senha" secureTextEntry />
```

As duas linhas fazem a mesma coisa. Em campo de senha, some com a sugestão do teclado: `autoCapitalize="none"` e `autoCorrect={false}`.

FECHANDO O TECLADO

`Keyboard.dismiss()`

```
import { Keyboard, TouchableWithoutFeedback } from 'react-native';

function fecharTeclado() {
  Keyboard.dismiss();
}

<TouchableWithoutFeedback onPress={fecharTeclado}>
  ...a tela inteira...
</TouchableWithoutFeedback>
```

JSX

No celular não existe "clique fora": o teclado fica aberto até alguém mandar fechar. Esse envelope é o padrão de toda tela com formulário.

KEYBOARDAVOIDINGVIEW

Para o teclado não cobrir o campo

```
<KeyboardAvoidingView
  style={styles.scrollview}
  behavior={Platform.OS == 'ios' ? 'padding' : 'height'}
>
  ...
</KeyboardAvoidingView>
```

JSX

- iOS: `behavior="padding"` — a tela ganha um espaço embaixo
- Android: `"height"` — a altura da tela é recalculada

`Platform.OS` devolve `'ios'` ou `'android'`. É assim que se escreve um comportamento diferente por plataforma sem duplicar o componente.

SCROLLVIEW NO FORMULÁRIO

`style` e `contentContainerStyle` não são a mesma coisa

```
<ScrollView
  contentContainerStyle={styles.scrollview} // estiliza o CONTEUDO
  bounces={false}
>
```

JSX

- `style` — estiliza a **caixa** que rola
- `contentContainerStyle` — estiliza o **conteúdo** que está dentro dela
- `flexGrow: 1` no conteúdo faz a tela ocupar tudo mesmo com pouco texto

`flex: 1` no `contentContainerStyle` trava a rolagem. O certo ali é `flexGrow: 1`.

A MOLDURA COMPLETA DO PROFILE

A ordem dos envelopes importa

JSX

```
<KeyboardAvoidingView behavior={Platform.OS == 'ios' ? 'padding' : 'height'}>
  <ScrollView contentContainerStyle={styles.scrollview} bounces={false}>
    <TouchableWithoutFeedback onPress={fecharTeclado}>
      <SafeAreaView style={styles.container}>
        <StatusBar style="auto" />
        {/* conteudo da tela */}
      </SafeAreaView>
    </TouchableWithoutFeedback>
  </ScrollView>
</KeyboardAvoidingView>
```

De fora para dentro: **desviar do teclado** → **poder rolar** → **fechar o teclado ao tocar** → **respeitar o entalhe**.

VALIDAÇÃO SIMPLES

O botão que só acende quando pode

```
<Button
  title="Salvar"
  onPress={submitForm}
  disabled={name.length == 0}
/>
```

JSX

```
function submitForm() {
  if (name.trim().length < 3) {
    return Alert.alert('Atenção', 'Nome muito curto.');
```

JSX

trim() evita que três espaços passem como nome válido.

PARTE 5

Fechamento e autoavaliação

SINTOMA E CAUSA

A tabela para consultar durante a prova prática

SINTOMA	CAUSA MAIS PROVÁVEL
Elementos empilhados	faltou <code>flexDirection: 'row'</code>
<code>justifyContent</code> não faz nada	falta <code>flex: 1</code> no container
A imagem não aparece	imagem remota sem <code>width</code> e <code>height</code>
A sombra sumiu no Android	falta <code>elevation</code>
O app caiu ao abrir a tela	texto solto fora de um <code><Text></code>
A tela não atualiza	estado alterado na mão, sem o <code>set</code>
O <code>Alert</code> aparece sozinho	<code>onPress={funcao()}</code> com parênteses

AS ARMADILHAS CAMPEÃS

Se você só decorar uma lista, decore esta

- `onPress={salvar}` — **sem** parênteses; com eles, roda na renderização
- `onChangeText` entrega o **texto**, não o evento
- Estilo **não se herda**: `color` vai no `<Text>`, nunca na `<View>`
- Array de estilo: **o último vence**
- `push` no estado não redesenha nada — crie um array novo
- `value` limpa o campo; `defaultValue` **não**

REGRA DA AULA A mensagem de erro é o começo da resposta, não o fim. Leia antes de apagar.

CHECKLIST DE VÉSPERA

Consegue fazer isto sem consultar?

- 1 Criar um componente com props e usá-lo em outra tela
- 2 Montar uma linha com ícone e texto alinhados e um botão à direita
- 3 Centralizar um bloco na tela inteira
- 4 Deixar um avatar redondo com imagem vinda da internet
- 5 Escrever um contador com `useState` e dois botões
- 6 Fazer um `TextInput` controlado que habilita um botão

Se travar em algum item, abra o `App.js` do `profile` : a resposta dos seis está lá dentro.

TESTE-SE — TEORIA (1)

Responda em voz alta antes de virar o slide

- 1 Qual a diferença entre React, React Native e Expo?
- 2 Por que `<View style={{ color: 'red' }}>` não pinta o texto de dentro?
- 3 Qual o `flexDirection` padrão no React Native, e por que isso importa?
- 4 Para que serve `flex: 1` no container da tela?
- 5 Quando usar `TouchableOpacity` em vez de `Button` ?

GABARITO — TEORIA (1)

Respostas curtas

- 1 React é a biblioteca de interface; React Native usa o React para controlar componentes **nativos**; Expo é o framework e os serviços em volta dele
- 2 Porque **não existe herança** de estilo: `color` só vale no `<Text>` que o recebeu
- 3 `column` — se você espera itens lado a lado sem declarar `row`, eles saem empilhados
- 4 Para a `View` ocupar todo o espaço disponível; sem isso, não sobra espaço para alinhar nada
- 5 Quando o botão precisa de estilo próprio — `Button` quase não aceita `style`

TESTE-SE — TEORIA (2)

A segunda rodada

- 1 O que acontece se você escrever `onPress={submitForm()}` ?
- 2 Por que `lista.push(item)` não atualiza a tela?
- 3 Qual a diferença prática entre `value` e `defaultValue` no `TextInput` ?
- 4 O que `disabled` faz no nosso componente `Button` ?
- 5 Por que `key` é obrigatória dentro de um `map` ?

GABARITO — TEORIA (2)

Respostas curtas

- 1 A função é chamada **durante a renderização**, e não no toque
- 2 `push` altera o array no lugar; a referência não muda e o React entende que nada mudou
- 3 `value` deixa o estado no comando (dá para limpar o campo); `defaultValue` só define o texto inicial
- 4 Duas coisas: bloqueia o toque no `TouchableOpacity` e reduz a opacidade para `0.3`
- 5 Para o React identificar cada item entre um redesenho e outro, sem recriar a lista inteira

MINI-DESAFIOS PRÁTICOS

Vinte minutos que valem por uma revisão inteira

- 1 Trocar o `defaultValue` do campo Nome por `value` e criar um botão "Limpar"
- 2 Extrair um componente `Input` com as props `placeholder`, `value` e `onChangeText`
- 3 Fazer o campo E-mail virar editável por um botão "Editar" que troca um estado
- 4 Adicionar contagem de caracteres embaixo do campo Nome
- 5 Trocar o texto "Alterar foto" por um ícone e um texto na mesma linha

Faça no próprio `profile`. Se quebrar, você aprendeu duas coisas em vez de uma.

COLINHA FINAL

O que costuma faltar na hora

JSX

```
// LAYOUT
flex: 1
flexDirection: 'row'
justifyContent: 'center'
alignItems: 'center'
gap: 16
paddingHorizontal: 24

// ESTILO
style={[base, condicao && extra]}
borderRadius: 999
elevation: 4
```

JSX

```
// ESTADO
const [v, setV] = useState('');
setV((prev) => prev + 1);
setLista([...lista, novo]);

// ENTRADA
value={v}
onChangeText={setV}
secureTextEntry
keyboardType="email-address"
Keyboard.dismiss();
```

RESUMO DA REVISÃO

O que precisa estar firme na hora da prova

- Componente é função que devolve JSX; **props descem, funções sobem**
- Estilo é objeto: camelCase, número puro, **sem herança**, o último do array vence
- Tudo é Flexbox e o padrão é **coluna**; `justifyContent` no eixo principal
- Toque é `onPress` ; escolha do componente de toque = escolha do **feedback**
- `useState` redesenha a tela; nunca altere o estado no lugar
- `TextInput` : `onChangeText` traz o texto, `value` controla, `Keyboard.dismiss` fecha

Boa prova. Reveja o `profile` uma vez inteiro, em voz alta, explicando cada linha.